

AgroSmart Andino

Guía de Implementación Completa
Pasos 1 al 8 — Local y AWS paso a paso

Pipeline Big Data con Dashboard en Tiempo Real

Monitoreo de Papa Nativa en Huangascar, Yauyos, Lima

Curso:	Big Data
Docente:	Mg. Rosa Virginia Encinas Quille
Universidad:	Universidad Nacional de Ingeniería (UNI)
Programa:	Maestría en Inteligencia Artificial
Grupo:	2
Integrantes:	Montes E. · Vizcardo F. · Massa Q. · Huali C. · Lazaro G.
Fecha:	Mayo 2026

Resumen del sistema:

Componente	Tecnología	Registros	Estado
Data Acquisition	Python 3.13	72 397	✓
Data Lake	AWS S3	3 CSV	✓
ETL + ML	Spark 3.4 / EMR	3 653 Gold	✓
Indexación	4 índices	18/47/31 días	✓
NoSQL	Cassandra 4.1	4 459 dry-run	✓
Dashboard	Streamlit 1.x	11 KPIs	✓
Informe PDF	LaTeX	20 páginas	✓
Presentación	Beamer	17 diapositivas	✓

Directorio base del proyecto:

d:\Documents\GitHub\Maestria_IA_caso_practico\Big_data\trabajo_final\agrosmart_andino\

Índice

1. Prerrequisitos e Instalación del Entorno	2
1.1. Software necesario (instalación local Windows)	2
1.2. Instalar dependencias Python	2
1.3. Estructura de directorios	2
2. Parte 1 – Estructura del Proyecto	3
3. Parte 2 – Adquisición y Validación de Datos	3
3.1. Script 01 – Descarga NASA POWER API	3
3.2. Script 02 – Generar datos IoT sintéticos	4
3.3. Script 03 – Preparar datos MIDAGRI	4
3.4. Script 04 – Validar los 3 datasets	4
4. Parte 3 – Data Lake en AWS S3	4
4.1. Paso previo: Crear el bucket S3 en AWS Console	5
4.2. Copiar credenciales al .env	5
4.3. Ejecutar la subida	5
5. Parte 4 – Pipeline ETL y ML con Apache Spark en EMR	6
5.1. Crear el clúster EMR	6
5.2. Conectarse a JupyterHub y ejecutar el notebook	6
5.3. Qué hace el notebook (38 celdas)	7
5.4. Ejecución local (alternativa sin EMR)	7
6. Parte 5 – Apache Cassandra (NoSQL)	7
6.1. Opción A: Cassandra local con Docker	8
6.2. Opción B: Cassandra en AWS Cloud9	9
6.3. Estructura de tablas Cassandra	9
7. Parte 6 – Dashboard Streamlit	9
7.1. Prerrequisitos	9
7.2. Ejecutar el dashboard	10
7.3. Componentes del dashboard (11 KPIs)	10
8. Parte 7 – Informe Técnico en LaTeX	11
9. Parte 8 – Presentación Beamer	11
9.1. Estructura de la presentación (guion 15 min)	12
10. Resumen: Comandos de Ejecución Completa	12
11. Solución de Problemas Frecuentes	13
12. Entregables Finales	13

1. Prerrequisitos e Instalación del Entorno

1.1 Software necesario (instalación local Windows)

Software	Versión probada	Instalación
Python (Miniconda)	3.13	conda install
MiKTeX	25.12	miktex.org (para PDF)
AWS CLI	2.x	aws.amazon.com/cli
Docker Desktop	24.x	Para Cassandra local (opcional)
VS Code	cualquiera	Editor recomendado

1.2 Instalar dependencias Python

Instalar paquetes Python

```
# Desde la carpeta agrosmart_andino/
conda activate base      # o el entorno de tu preferencia
pip install -r requirements.txt
```

Contenido de requirements.txt:

```
pandas>=2.0
numpy>=1.24
requests>=2.31
boto3>=1.34
python-dotenv>=1.0
streamlit>=1.30
plotly>=5.18
cassandra-driver>=3.29
pyspark>=3.4      # solo necesario si ejecutas localmente sin EMR
```

1.3 Estructura de directorios

```
agrosmart_andino/
+-- data/
|   +-- raw/          <- CSVs descargados (Parte 2)
|   +-- gold/         <- CSVs procesados para dashboard (Parte 4+5)
+-- scripts/
|   +-- data_acquisition/
|   |   +-- 01_download_nasa_power.py
|   |   +-- 02_generate_iot_data.py
|   |   +-- 03_prepare_midagri_data.py
|   |   +-- 04_validate_datasets.py
|   +-- aws/
|   |   +-- 05_upload_to_s3.py
|   |   +-- 06_create_emr_cluster.py
|   +-- cassandra/
|   |   +-- 07_create_schema.cql
|   |   +-- 08_load_data.py
|   |   +-- 09_setup_cassandra_cloud9.sh
+-- notebooks/
|   +-- agrosmart_pipeline.ipynb    <- Spark ETL + ML (38 celdas)
+-- dashboard/
|   +-- app.py                    <- Streamlit dashboard
+-- docs/
|   +-- informe_tecnico.tex/.pdf    <- Informe (20 pags)
```

```
|   +-- presentacion.tex/.pdf      <- Diapositivas (17 slides)
|   +-- compilar.bat
+-- .env                          <- Credenciales AWS (NO subir a git)
+-- .env.example
+-- requirements.txt
+-- README.md
```

2. Parte 1 – Estructura del Proyecto

Objetivo: Crear la estructura de carpetas y archivos de configuración base.

Crear la estructura de carpetas

```
# Crear directorios desde trabajo_final/
mkdir -p agrosmart_andino/data/raw
mkdir -p agrosmart_andino/data/gold
mkdir -p agrosmart_andino/scripts/data_acquisition
mkdir -p agrosmart_andino/scripts/aws
mkdir -p agrosmart_andino/scripts/cassandra
mkdir -p agrosmart_andino/notebooks
mkdir -p agrosmart_andino/dashboard
mkdir -p agrosmart_andino/docs
```

Configurar archivo .env

```
# Copiar plantilla y editar con tus credenciales AWS
copy agrosmart_andino\.env.example agrosmart_andino\.env
# Editar .env y completar:
#   AWS_ACCESS_KEY_ID=ASIA...
#   AWS_SECRET_ACCESS_KEY=...
#   AWS_SESSION_TOKEN=...
#   S3_BUCKET_NAME=agrosmart-andino-demo
#   AWS_DEFAULT_REGION=us-east-1
```

Resultado Parte 1

Estructura de carpetas creada. Archivos: README.md, requirements.txt, .gitignore, .env.example.

3. Parte 2 – Adquisición y Validación de Datos

Objetivo: Descargar y generar los 3 datasets fuente. Validar 72 397 registros.

3.1 Script 01 – Descarga NASA POWER API

Ejecutar desde agrosmart_andino/

```
cd agrosmart_andino
python scripts/data_acquisition/01_download_nasa_power.py
```

Qué hace: Llama la API gratuita de NASA POWER (<https://power.larc.nasa.gov/api/temporal/daily/>) para 5 estaciones en Yauyos. Descarga 10 años de datos diarios (2015–2024).

Resultado esperado

data/raw/nasa_power_yauyos.csv — 18 265 filas, 1.6 MB
5 estaciones: Huangascar, Yauyos, Carania, Tanta, Vilca

Si hay error de red

La API NASA POWER es gratuita y no requiere clave. Si hay timeout, ejecutar nuevamente.
La URL base es: <https://power.larc.nasa.gov/api/temporal/daily/point>

3.2 Script 02 – Generar datos IoT sintéticos

```
python scripts/data_acquisition/02_generate_iot_data.py
```

Qué hace: Genera datos horarios de 3 nodos IoT simulados para las coordenadas de Huangascar. Usa distribuciones calibradas con el clima real de la zona.

data/raw/sensores_iot.csv — 52 632 filas, 4.6 MB
3 nodos: NODO-001 (3 000m), NODO-002 (3 200m), NODO-003 (3 500m)

3.3 Script 03 – Preparar datos MIDAGRI

```
python scripts/data_acquisition/03_prepare_midagri_data.py
```

data/raw/produccion_papa_yauyos.csv — 1 500 filas
Producción histórica de papa nativa por distrito y variedad (2000–2024)

3.4 Script 04 – Validar los 3 datasets

```
python scripts/data_acquisition/04_validate_datasets.py
```

Qué valida: Existencia, filas mínimas, duplicados, nulos críticos, rangos de variables (temperatura, humedad, precipitación).

```
[OK] nasa_power_yauyos.csv      -- 18,265 filas  -- VALIDO
[OK] sensores_iot.csv          -- 52,632 filas  -- VALIDO
[OK] produccion_papa_yauyos.csv -- 1,500 filas  -- VALIDO
Total: 72,397 registros validados
```

4. Parte 3 – Data Lake en AWS S3

Objetivo: Subir los 3 CSVs validados a un bucket S3 como Data Lake.

4.1 Paso previo: Crear el bucket S3 en AWS Console

AWS Console – Crear bucket S3

1. Iniciar sesión en AWS Academy
2. Abrir el Learner Lab → **[Start Lab]** → esperar indicador VERDE
3. Clic en **[AWS]** para abrir la consola
4. Buscar **S3** → **Create bucket**
5. Configurar:
 - Bucket name: **agrosmart-andino-demo**
 - Region: **US East (N. Virginia) us-east-1**
 - Block Public Access: activado (dejar por defecto)
 - Versioning: Disabled
6. Clic **Create bucket**

4.2 Copiar credenciales al .env

AWS Academy – Copiar credenciales

1. En Learner Lab → **AWS Details**
2. Copiar los 3 valores y pegarlos en `.env`:

```
AWS_ACCESS_KEY_ID=ASIA...
AWS_SECRET_ACCESS_KEY=...
AWS_SESSION_TOKEN=IQoJ... <- token largo, NO OMITIR
S3_BUCKET_NAME=agrosmart-andino-demo
AWS_DEFAULT_REGION=us-east-1
```

Las credenciales de AWS Academy expiran en 3 horas

Cada vez que reinicies el lab debes copiar nuevas credenciales. El `AWS_SESSION_TOKEN` es obligatorio; sin él la conexión falla.

4.3 Ejecutar la subida

Desde `agrosmart_andino/`

```
python scripts/aws/05_upload_to_s3.py
```

```
[OK] s3://agrosmart-andino-demo/raw/clima/nasa_power_yauyos.csv
[OK] s3://agrosmart-andino-demo/raw/sensores/sensores_iot.csv
[OK] s3://agrosmart-andino-demo/raw/produccion/produccion_papa_yauyos.csv
Total almacenado: 6.3 MB
```

5. Parte 4 – Pipeline ETL y ML con Apache Spark en EMR

Objetivo: Crear clúster EMR, ejecutar el notebook PySpark con ETL + índices agroclimáticos + modelos ML.

5.1 Crear el clúster EMR

Opción A: Script automático

```
# Desde agrosmart_andino/ (necesita credenciales en .env)
python scripts/aws/06_create_emr_cluster.py
# Esperar ~8-10 minutos hasta estado WAITING
```

Opción B: AWS Console manualmente

1. AWS Console → **EMR** → **Create cluster**
2. Configuración:
 - Nombre: **AgroSmart-Cluster**
 - EMR release: **emr-6.15.0**
 - Applications: **Spark, JupyterHub**
 - Primary (master): **m5.xlarge**
 - Core: **m5.xlarge** (1 nodo)
 - Key pair: crear o usar existente
3. Clic **Create cluster** → esperar estado **Waiting**

5.2 Conectarse a JupyterHub y ejecutar el notebook

Acceder a JupyterHub en EMR

1. EMR Console → tu clúster → pestaña **Applications**
2. Copiar la URL de JupyterHub (puerto 9443)
3. Abrir en navegador, usuario: **jovyan** / pass: **jupyter**
4. Subir el archivo **notebooks/agrosmart_pipeline.ipynb**
5. Menú: **Kernel** → **PySpark** → ejecutar todas las celdas

5.3 Qué hace el notebook (38 celdas)

Sección	Celdas	Acción
Setup SparkSession	1–3	Inicializar Spark con S3 como storage
Ingesta S3 Raw	4–7	Leer 3 CSV, revisar schema y nulos
Limpieza ETL	8–13	Dedup, imputación, tipado, fechas
Join e integración	14–17	Unir clima + IoT + producción
Índices agroclimáticos	18–22	Calcular IHB, ITT, IEH, ET ₀
EDA	23–26	Estadísticas descriptivas, correlación
RandomForest (ML)	27–30	Clasificación riesgo tizón, AUC=0.87
GBT Regressor (ML)	31–34	Predicción rendimiento, R ² =0.78
Exportar Gold	35–38	Escribir 4 CSV Gold a S3 Gold

Archivos Gold generados en S3

```
s3://agrosmart-andino-demo/gold/lecturas_climaticas.csv      # 3,653
    filas
s3://agrosmart-andino-demo/gold/alertas_activas.csv          # 500 filas
s3://agrosmart-andino-demo/gold/recomendaciones_riego.csv     # 300 filas
s3://agrosmart-andino-demo/gold/predicciones_rendimiento.csv # 6 filas
```

Descargar los CSV Gold para uso local

Después de ejecutar el notebook, descargar los 4 CSV Gold a `data/gold/` para poder usar el dashboard localmente sin EMR. Desde AWS Console → S3 → gold/ → Download.

5.4 Ejecución local (alternativa sin EMR)

Si tienes PySpark instalado localmente

```
# Instalar PySpark
pip install pyspark

# Ejecutar el notebook localmente
cd agrosmart_andino
jupyter notebook notebooks/agrosmart_pipeline.ipynb
# NOTA: cambiar la ruta de S3 por la ruta local data/raw/
# "s3://..." -> "data/raw/..."
```

6. Parte 5 – Apache Cassandra (NoSQL)

Objetivo: Crear el esquema Cassandra y cargar los datos Gold.

6.1 Opción A: Cassandra local con Docker

Docker Desktop – Instalar Cassandra 4.1

```
# Descargar imagen y ejecutar contenedor
docker pull cassandra:4.1
docker run --name cassandra-agrosmart \
  -p 9042:9042 \
  -e HEAP_NEWSIZE=128M \
  -e MAX_HEAP_SIZE=512M \
  -d cassandra:4.1

# Esperar ~30 segundos a que arranque
docker logs cassandra-agrosmart --tail 20
# Debe aparecer: "Starting listening for CQL clients on /0.0.0.0:9042"
```

Crear el esquema CQL

```
# Copiar el schema al contenedor y ejecutar
docker cp scripts/cassandra/07_create_schema.cql cassandra-agrosmart:/tmp/

docker exec -it cassandra-agrosmart \
  cqlsh -f /tmp/07_create_schema.cql

# Verificar keyspace creado
docker exec -it cassandra-agrosmart \
  cqlsh -e "DESCRIBE KEYSPACES;"
# Debe aparecer: agrosmart_andino
```

Cargar datos Gold en Cassandra

```
# Ejecutar desde agrosmart_andino/
# Modo dry-run (sin conexión real, solo valida datos):
python scripts/cassandra/08_load_data.py --dry-run

# Modo real (requiere Cassandra corriendo en localhost:9042):
python scripts/cassandra/08_load_data.py
```

```
[DRY-RUN] lecturas_climaticas : 3,653 filas
[DRY-RUN] alertas_activas      : 500   filas
[DRY-RUN] recomendaciones_riego : 300   filas
[DRY-RUN] predicciones_rendimiento: 6    filas
Total: 4,459 filas procesadas en 0.1s (79,917 filas/seg)
```

6.2 Opción B: Cassandra en AWS Cloud9

AWS Cloud9 – Instalar Docker + Cassandra

```
# 1. Crear entorno Cloud9 en AWS Console
#     Tipo: EC2 t3.medium, Amazon Linux 2023

# 2. En la terminal de Cloud9, ejecutar el script:
bash scripts/cassandra/09_setup_cassandra_cloud9.sh

# El script automaticamente:
#   - Instala Docker
#   - Descarga cassandra:4.1
#   - Ejecuta el contenedor
#   - Crea el keyspace agrosmart_andino
#   - Crea las 5 tablas
```

Compatibilidad Python 3.13 con cassandra-driver

En Python 3.13 el módulo `asyncore` fue eliminado. El script `08_load_data.py` ya incluye el fix:

```
from cassandra.io.asyncioreactor import AsyncioConnection
cluster = Cluster(
    ['localhost'],
    connection_class=AsyncioConnection
)
```

No cambiar esto; es obligatorio para Python 3.13.

6.3 Estructura de tablas Cassandra

Tabla	Partition Key	TTL
lecturas_climaticas_por_dia	(estacion, fecha DESC)	Sin TTL
alertas_activas	(nodo_id, fecha DESC)	90 días
recomendaciones_riego	(nodo_id, fecha DESC)	90 días
predicciones_rendimiento	(anio, variedad)	Sin TTL
estado_nodos	(nodo_id)	Sin TTL

7. Parte 6 – Dashboard Streamlit

Objetivo: Ejecutar el dashboard interactivo de monitoreo en tiempo real.

7.1 Prerrequisitos

Antes de ejecutar el dashboard, verificar que existan los 4 CSV Gold en `data/gold/`:

```
data/gold/
+-- lecturas_climaticas.csv          (3,653 filas)
+-- alertas_activas.csv              (500 filas)
+-- recomendaciones_riego.csv        (300 filas)
+-- predicciones_rendimiento.csv     (6 filas)
```

Si no existen, copiarlos desde S3 o ejecutar las Partes 2–4 primero.

7.2 Ejecutar el dashboard

IMPORTANTE: ejecutar DESDE la carpeta `agrosmart_andino/`

```
# Navegar a la carpeta correcta
cd d:\Documents\GitHub\Maestria_IA_caso_practico\Big_data\trabajo_final\
  agrosmart_andino

# Ejecutar Streamlit
streamlit run dashboard/app.py --server.port 8501
```

Error frecuente: File does not exist: `dashboard\app.py`

Causa: El comando se ejecuta desde la carpeta equivocada.

Solución: Siempre ejecutar desde `agrosmart_andino/`, no desde `trabajo_final/`.

```
# INCORRECTO (desde trabajo_final/):
streamlit run dashboard/app.py

# CORRECTO (desde agrosmart_andino/):
cd agrosmart_andino
streamlit run dashboard/app.py --server.port 8501
```

Dashboard corriendo

```
You can now view your Streamlit app in your browser.
Local URL: http://localhost:8501
Network URL: http://192.168.x.x:8501
```

Abrir `http://localhost:8501` en el navegador.

7.3 Componentes del dashboard (11 KPIs)

#	Componente	Descripción
1	Semáforo HTML	Verde/Naranja/Rojo según nivel de alerta
2	KPI Nivel alerta	Texto coloreado con el estado del nodo
3	KPI Humedad suelo	Porcentaje medio del nodo seleccionado
4	KPI Recomendación	Texto de acción de riego
5	Tabla alertas	Últimas 10 alertas por nodo
6	Métricas 7 valores	T_{med} , $T_{mín}$, ET_0 , déficit, horas riego, etc.
7	Temperatura 7 días	Línea máx/med/mín con zona de helada
8	Precipitación 30 d	Barras diarias
9	Humedad suelo	Línea por nodo con bandas de estrés
10	Heatmap ITT	Matriz $sem \times día$ verde a rojo
11	Gauge + barras GBT	Predicción rendimiento 2025 por variedad

Controles del sidebar:

- Selector de nodo: NODO-001 / NODO-002 / NODO-003
- Slider historial: 7 a 90 días
- Botón *Recargar datos*: fuerza recarga de CSV
- Refresco automático cada 5 minutos (@st.cache_data(ttl=300))

8. Parte 7 – Informe Técnico en LaTeX

Objetivo: Compilar el informe técnico PDF de 20 páginas.

Compilar el informe (dos pasadas)

```
cd agrosmart_andino\docs

# Primera pasada (genera estructura)
pdflatex -interaction=nonstopmode informe_tecnico.tex

# Segunda pasada (resuelve referencias cruzadas e indice)
pdflatex -interaction=nonstopmode informe_tecnico.tex

# O usar el script batch incluido:
compilar.bat
```

Warning "Incompatible glue units"(x4)

Este warning proviene de `tcolorbox` + `babel/spanish`. Es inofensivo; el PDF se genera correctamente (20 páginas). No es un error que impida la compilación.

docs/informe_tecnico.pdf — 20 páginas, ≈491 KB
Secciones: Portada, Resumen, 12 secciones técnicas, Referencias, Anexos A/B/C

9. Parte 8 – Presentación Beamer

Objetivo: Compilar las diapositivas Beamer PDF de 17 slides.

Compilar la presentación

```
cd agrosmart_andino\docs

# Dos pasadas para resolver el indice de diapositivas
pdflatex -interaction=nonstopmode presentacion.tex
pdflatex -interaction=nonstopmode presentacion.tex
```

docs/presentacion.pdf — 17 diapositivas, ≈495 KB
Duración estimada: 15 minutos + 5 minutos Q&A

9.1 Estructura de la presentación (guion 15 min)

Slide	Sección	Min	Puntos clave
1	Portada	0:00	Título, integrantes, docente
2	Índice	0:30	Navegación de la presentación
3	Problema	1:30	3 riesgos, pérdida 38 %, oportunidad
4	Objetivos	3:00	General + 5 específicos con métricas
5	Arquitectura	4:30	Flujo 5 capas, tabla tecnologías
6	Datos	6:00	72 397 registros, 3 fuentes, validación
7	ETL Spark	7:30	3 fases, config EMR, tabla resultados
8	Índices	8:30	Fórmulas IHB/ITT/IEH/ET ₀
9	ML	10:00	RF AUC=0.87, GBT R ² =0.78
10	Predicciones	11:00	6 variedades, 9.2 t/ha 2025
11	Cassandra	12:00	5 tablas, dry-run, design patterns
12	Dashboard	13:00	11 KPIs, ejecución, semáfor
13	Semáforos	13:30	Mock 3 nodos verde/naranja/rojo
14	Resultados	14:00	Tabla global de métricas
15	Contribuciones	14:30	4 aportes + limitaciones
16	Conclusiones	15:30	6 conclusiones enumeradas
17	Cierre Q&A	16:00	Gracias + preguntas

10. Resumen: Comandos de Ejecución Completa

A continuación, el orden completo de ejecución para reproducir todo el proyecto desde cero:

```
# == PASO 0: Instalar dependencias =====
cd agrosmart_andino
pip install -r requirements.txt

# == PASO 1: Adquirir datos (Parte 2) =====
python scripts/data_acquisition/01_download_nasa_power.py
python scripts/data_acquisition/02_generate_iot_data.py
python scripts/data_acquisition/03_prepare_midagri_data.py
python scripts/data_acquisition/04_validate_datasets.py

# == PASO 2: Subir a S3 (Parte 3) -- requiere AWS activo ====
# Copiar credenciales de AWS Academy en .env
python scripts/aws/05_upload_to_s3.py

# == PASO 3: ETL + ML en EMR (Parte 4) =====
# Crear cluster EMR:
python scripts/aws/06_create_emr_cluster.py
# Abrir JupyterHub y ejecutar:
# notebooks/agrosmart_pipeline.ipynb
# Descargar 4 CSV Gold a data/gold/

# == PASO 4: Cassandra (Parte 5) =====
docker run --name cassandra-agrosmart -p 9042:9042 \
  -e MAX_HEAP_SIZE=512M -d cassandra:4.1
# esperar 30 segundos...
```

```

docker cp scripts/cassandra/07_create_schema.cql cassandra-agrosmart:/tmp/
docker exec -it cassandra-agrosmart cqlsh -f /tmp/07_create_schema.cql
python scripts/cassandra/08_load_data.py --dry-run      # validar
python scripts/cassandra/08_load_data.py                # cargar real

# == PASO 5: Dashboard (Parte 6) =====
# IMPORTANTE: ejecutar desde agrosmart_andino/, no desde trabajo_final/
cd agrosmart_andino
streamlit run dashboard/app.py --server.port 8501
# Abrir: http://localhost:8501

# == PASO 6: Compilar documentacion (Partes 7 y 8) =====
cd docs
pdflatex -interaction=nonstopmode informe_tecnico.tex
pdflatex -interaction=nonstopmode informe_tecnico.tex
pdflatex -interaction=nonstopmode presentacion.tex
pdflatex -interaction=nonstopmode presentacion.tex

```

11. Solución de Problemas Frecuentes

Error	Causa	Solución
File does not exist: dashboard\app.py	Ejecutar desde carpeta incorrecta	cd agrosmart_andino antes de streamlit
NoCredentialsError	.env vacío o no encontrado	Copiar credenciales de AWS Academy Details
asyncore not found	Python 3.13 eliminó asyncore	Ya corregido: usar AsyncioConnection
applymap deprecated	Pandas ≥ 2.1 ren. applymap	Ya corregido: usar .style.map()
Incompatible glue units (LaTeX)	tclobx + babel/spanish	Es un warning inofensivo, el PDF se genera
EMR cluster en estado TERMINATED	Sesión AWS de 3h expiró	Reiniciar lab y crear nuevo cluster
Cassandra: Connection refused	Docker no iniciado	docker start cassandra-agrosmart
NASA POWER: timeout	API lenta o red	Ejecutar nuevamente (es gratuita, sin clave)

12. Entregables Finales

Archivo	Descripción	Tamaño
docs/informe_tecnico.pdf	Informe técnico completo	20 pág., 491 KB
docs/presentacion.pdf	Diapositivas Beamer	17 slides, 495 KB
dashboard/app.py	Código fuente dashboard	Streamlit
notebooks/agrosmart_pipeline.ipynb	Notebook Spark EMR	38 celdas
data/gold/*.csv	Datasets procesados	4 archivos
scripts/	Todos los scripts (01–09)	9 archivos

Métricas clave para mencionar en la presentación

- 72 397 registros, 3 fuentes
- 3 653 registros Gold integrados
- AUC-ROC = **0.87** (RandomForest)
- $R^2 = 0.78$ (GBT Regressor)
- 9.2 t/ha predicción 2025
- 18 días helada / 47 tizón / 31 hídrico
- 4 459 filas Cassandra en 0.1 s
- <3 min ETL en EMR 6.15
- 11 componentes Streamlit
- <\$50/mes costo AWS estimado